SUBIECTE TEST1 BIA INTORTOCHEATE

1. Use the following code to generate a classification dataset:

```
X, y = make_classification (
        n_samples=1000,
        n_features=4,
        n_informative=4,
        n_redundant=0,
        n_classes=3,
        n_clusters_per_class=1,
        random_state=42
}
```

Split the data into a train and test split, the test split being 25% of the initial dataset.
Apply a gradient boosting ensemble formed of 200 decision trees with maximum depth of 3, with a learning rate of 1.2, on the train set.
Compute the feature importances, the accuracy on the test set and the output probability distribution of the 11th sample in the test set.
(array([0.13183207, 0.18527228, 0.34405552,
0.33884012]), 0.892, array([[8.55051214e-14, 2.46194816e-14, 1.00000000e+00]]))

REZOLVARE:

```
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import accuracy_score

# Generate the dataset
X, y = make_classification(n_samples=1000, n_features=4, n_informative=4,
n_redundant=0, n_classes=3, n_clusters_per_class=1, random_state=42)

# Split the dataset into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
random_state=42)

# Apply a gradient boosting ensemble formed of 200 decision trees with max
imum depth of 3 and a learning rate of 1.2 on the train set
gb = GradientBoostingClassifier(n_estimators=200, max_depth=3, learning_ra
te=1.2, random_state=42)
gb.fit(X_train, y_train)

# Compute the feature importances
feature_importances = gb.feature_importances_
```

```
# Compute the accuracy on the test set
y_pred = gb.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)

# Compute the output probability distribution of the 11th sample in the te
st set
sample_11_probabilities = gb.predict_proba([X_test[10]])[0]

# Print the results
print("Feature importances:", feature_importances)
print("Accuracy on the test set:", accuracy)
print("Output probability distribution of the 11th sample in the test set:
", sample_11_probabilities)
```

2. Use the following code to generate a classification dataset:

X, y = make_classification (
n_samples=1000,
factors=0.5,
noise=0.1,
random_state=42
}

Split the data into a train and test split, the test split being 25% of the initial dataset.
Apply the K-means clustering algorithm with K=3, on the train set.
Compute the label of the 11th sample in the test set, the first cluster center, the inertia on the train set, the negative inertia on the test set, and the normalized mutual information score.
(2, array([0.24530604, 0.53353481]), 186.75691886796457,

-58.722482875379455, 0.0023975255667463614)

REZOLVARE:

```
from sklearn.datasets import make_circles
from sklearn.model_selection import train_test_split
from sklearn.cluster import KMeans
from sklearn.metrics import normalized_mutual_info_score

# Generate the dataset
X, y = make_circles(n_samples=1000, factor=0.5, noise=0.1, random_state=42
)

# Split the dataset into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
random_state=42)

# Apply the K-means clustering algorithm with K=3 on the train set
```

```
kmeans = KMeans(n_clusters=3, random_state=42)
kmeans.fit(X_train)

# Compute the label of the 11th sample in the test set
sample_11_label = kmeans.predict([X_test[10]])[0]

# Compute the first cluster center
cluster_centers = kmeans.cluster_centers_
first_cluster_center = cluster_centers[0]

# Compute the inertia on the train set
train_inertia = kmeans.inertia_

# Compute the negative inertia on the test set
test_negative_inertia = -kmeans.score(X_test)

# Compute the normalized mutual information score
normalized_mutual_information = normalized_mutual_info_score(y_test, kmean
s.predict(X_test))

# Print the results
print("Label of the 11th sample in the test set:", sample_11_label)
print("First cluster center:", first_cluster_center)
print("Inertia on the train set:", train_inertia)
print("Negative inertia on the test set:", test_negative_inertia)
print("Normalized mutual information score:", normalized_mutual_informatio
n)
```

3. Use the following code to generate a regression dataset:

```
X, y = make_classification (
        n_samples=1000,
        n_features=2,
        noise=1,
        bias=2,
        random_state=42
}
```

Split the data into a train and test split being 25% of the initial dataset.
Apply the polynomial kernel SVM algorithm with degree 2 coefficient 0.5 C=2 and ε=1 for 5000 epoches on the train set.
Compute the second support vector and the mean squared error on the test set.

(array([0.11422765, 0.15030176 nu sunt sigura]), 0.9818168401064361]

REZOLVARE:

```
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split
from sklearn.svm import SVR
from sklearn.metrics import mean_squared_error

# Generate the dataset

X, y = make_regression(n_samples=1000, n_features=2, noise=1, bias=2, rand
om_state=42)

# Split the dataset into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
random_state=42)

# Apply the polynomial kernel SVM algorithm with degrees 2, coefficient 0.
5, C=2, and c=1 for 5000 epochs on the train set
svm = SVR(kernel='poly', degree=2, coef0=0.5, C=2, epsilon=1, max_iter=500
0)
svm.fit(X_train, y_train)

# Compute the second support vector
support_vectors = svm.support_vectors_
second_support_vector = support_vectors[1]

# Compute the mean squared error on the test set
y_pred = svm.predict(X_test)
test_mse = mean_squared_error(y_test, y_pred)

# Print the results
print("Second support vector:", second_support_vector)
print("Mean squared error on the test set:", test_mse)
```

4. Use the following code to generate a regression dataset:

```
X, y = make_regression (
        n_samples=1000,
        n_features=1,
        noise=0.5,
        bias=3,
        random_state=42
}
```

Split the data into a train and test split being 15% of the initial dataset.
Apply the stochastic gradient descent regression algorithm with l2 regularization with λ=0.01 for 150 epochs with a con... learning rate.
n=0.005 on the train set
Compute w, w0, and the mean squared error on the test set.
(array([16.5730716]), array([2.96446862]), 0.2826888309401727)

```
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split
from sklearn.linear_model import SGDRegressor
from sklearn.metrics import mean_squared_error

# generate dataset
X, y = make_regression(n_samples=1000, n_features=1, noise=0.5, bias=3, ra
ndom_state=42)

# split dataset into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.15,
random_state=42)

# apply stochastic gradient descent regression algorithm
sgd reg = SGDRegressor(penalty='12', alpha=0.01, max iter=150, eta0=0.005,
 random_state=42)
sgd_reg.fit(X_train, y_train)

# compute omega and omega0
omega = sgd reg.coef
omega0 = sgd_reg.intercept_

# compute mean squared error on the test set
y_pred = sgd_reg.predict(X_test)
mse = mean squared error(y test, y pred)

print("Omega:", omega)
print("Omega0:", omega0)

 print("Mean squared error on test set:", mse)
```

5. Use the following code to generate a classification dataset:

```
X, y = make_classification (
        n_samples=1000,
        n_features=4,
        n_informative=4,
        n_redundant=0,
        n_classes=3,
        n_clusters_per_class=1,
        random_state=42
}
```

Split the data intor a train and test split, the test split being 25% of the initial dataset.

Apply a soft voting ensemble formed of a logistic regression algorithm, a random forest algorithm with 200 estimators and a Gaussian RGB kernel (nu sunt sigura la RGB kernel) SVM algorithm with y=1.5 on the train set.

Compute the accuracy on the test set and the output probability distribution of the 9th sample in the test set.

(0.912, array([[0.57070994, 0.21619792, 0.21309213]]))

REZOLVARE

```
from sklearn.datasets import make_classification


from sklearn.model_selection import train_test_split
from sklearn.ensemble import VotingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy score

# Generate the dataset
X, y = make_classification(n_samples=1000, n_features=4, n_informative=4,
n redundant=0, n classes=3, n clusters per class=1, random state=42)

# Split the dataset into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
random_state=42)

# Define the three models to include in the voting ensemble
lr = LogisticRegression(random_state=42)
rf = RandomForestClassifier(n_estimators=200, random_state=42)
svm = SVC(kernel='rbf', gamma=1.5, probability=True, random_state=42)

# Define the voting classifier with 'soft' voting
voting_clf = VotingClassifier(estimators=[('lr', lr), ('rf', rf), ('svm',
svm)], voting='soft')

# Train the voting classifier on the train set
voting_clf.fit(X_train, y_train)

# Compute the accuracy on the test set
test accuracy = voting clf.score(X test, y test)

# Compute the output probability distribution for the 9th sample in the te
st set
sample index = 8 # Python uses 0-based indexing
sample = X_test[sample_index]
sample proba = voting clf.predict proba([sample])[0]

# Print the results
print("Accuracy on the test set:", test_accuracy)
print("Output probability distribution for the 9th sample in the test set:
", sample_proba)
```

6. Use the following code to generate a classification dataset:

```
X, y = make_circles (
        n_samples=1000,
        factor =0.1,
```

noise=0.01,
random_state=42
}
Split the data into a train and test split being 25% of the initial dataset.
Apply the PCA algorithm with 2 components on the train set.
Compute the components, the explained variance ratio and the inverse transformation of the 11th sample in the test(?) set.

(array([[-0.76930085, 0.63888669], [0.63888669, 0.76930009?]]), array([0.50537804, 0.496621962?),
array([-0.5139503, 0.82538319]))

REZOLVARE:

```
from sklearn.datasets import make_circles
from sklearn.model_selection import train_test_split
from sklearn.decomposition import PCA

# Generate the dataset
X, y = make_circles(n_samples=1000, factor=0.1, noise=0.01, random_state=4
2)

# Split the dataset into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
random_state=42)

# Apply PCA with 2 components on the train set
pca = PCA(n_components=2)
X_train_pca = pca.fit_transform(X_train)

# Print the components
print("Components:\n", pca.components_)

# Print the explained variance ratio
print("Explained Variance Ratio:", pca.explained_variance_ratio_)

# Inverse transform the 11th sample in the test set using the PCA transfor
mation
X_test_pca = pca.transform(X_test)
sample_11 = X_test_pca[10]
sample_11_inv = pca.inverse_transform(sample_11.reshape(1, -1))

# Print the inverse transformed sample
print("Inverse Transformed Sample:\n", sample_11_inv)
```

7. Use the following code to generate an S-shaped curve dataset:

```
X, y = make_s_curve(
    n_samples=1000,
    random_state=42
)
```

Split the data into a train and test split, the test split being 25% of the initial dataset.

Apply the locally linear embedding algorithm with 3 components and 6 neighbors, on the train set.

Compute the transformation of the 11th sample in the test set.

array([-0.04161236, 0.04043414, 0.01098724])

REZOLVARE:

```
from sklearn.datasets import make_s_curve
from sklearn.model_selection import train_test_split
from sklearn.manifold import LocallyLinearEmbedding

# generate the dataset
X, y = make_s_curve(n_samples=1000, random_state=42)

# split into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
random_state=42)

# apply locally linear embedding with 3 components and 6 neighbors
lle = LocallyLinearEmbedding(n_components=3, n_neighbors=6)
X_train_lle = lle.fit_transform(X_train)

# transform the 11th sample in the test set
X_test_11_lle = lle.transform(X_test[10:11])

# print the transformed sample
print("Transformed 11th sample in the test set:", X_test_11_lle)
```

8. Use the following code to generate a classification dataset:

X, y = make_circles (
n_samples=1000,
factor=0.5,
noise=0.1,
random_state=42
}

Split the data into a train and test split, the test split being 25% of the initial dataset.
Apply the K-means clustering algorithm with K=4, random initialization, and 15 consecutive initializations on the train set.
Compute the label of the 9th sample in the test set, the second cluster center, the inertia on the train set, the negative inertia on the test set, and the adjusted Rand score.
(1, array([-0.6731441, -0.07718862]), 132.22416031964735,

-40.94325371413339, -0.0031247277429594074)

REZOLVARE:

```
from sklearn.datasets import make_circles
from sklearn.model_selection import train_test_split
from sklearn.cluster import KMeans
from sklearn.metrics import adjusted_rand_score

X, y = make_circles(n_samples=1000, factor=0.5, noise=0.1, random_state=42
)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
random_state=42)

kmeans = KMeans(n_clusters=4, init='random', n_init=15, random_state=42)
kmeans.fit(X_train)

label_9th = kmeans.predict([X_test[8]])[0]
cluster_center_2nd = kmeans.cluster_centers_[1]
inertia_train = kmeans.inertia_
negative_inertia_test = -kmeans.score(X_test)
adjusted_rand = adjusted_rand_score(y_test, kmeans.predict(X_test))

print("Label of the 9th sample in the test set:", label_9th)
print("Second cluster center:", cluster_center_2nd)
print("Inertia on the train set:", inertia_train)
print("Negative inertia on the test set:", negative_inertia_test)
print("Adjusted Rand score:", adjusted_rand)
```

9. Use the following code to generate a classification dataset:

X, y = make_circles (
n_samples=1000,

factor=0.1
noise=0.01,
random_state=42
}

Split the data into a train and test split, the test split being 25% of the initial dataset.
Apply the PCA algorithm with 2 components, on the train set.
Compute the components, the explained variance ratio, and the inverse transformation of the 11th sample in the test set.

(array([[-0.76930085, 0.63888663], [0.63888669, 0.76930085]]), array([0.50537804?, 0. .....)), array([-0.5139503, 0.82538319]))

REZOLVARE:

```
from sklearn.datasets import make_circles
from sklearn.model_selection import train_test_split
from sklearn.decomposition import PCA

# Generate dataset
X, y = make_circles(n_samples=1000, factor=0.1, noise=0.01, random_state=4
2)

# Split into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
random_state=42)

# Apply PCA with 2 components
pca = PCA(n_components=2)
X_train_pca = pca.fit_transform(X_train)

# Compute the components
components = pca.components_

# Compute the explained variance ratio
explained_variance_ratio = pca.explained_variance_ratio_

# Compute the inverse transformation of the 11th sample in the test set
X_test_pca_inv = pca.inverse_transform(X_test[10].reshape(1,-1))

print("Components:")
print(components)
print("Explained Variance Ratio:")
print(explained_variance_ratio)
print("Inverse Transformation of 11th sample in test set:")
print(X_test_pca_inv)
```

10. Use the following code to generate a regression dataset:

X, y = make_regression (
n_samples=1000,
n_features=1,
noise=2,
bias=3,
random_state=42
}

Split the data into a train and test split being 20% of the initial dataset.

Apply the ridge regression algorithm with λ = 0.2 on the train set.

Compute w, w0, and the mean squared error on the test set.

(array([16.73745169]), 2.981569554379661?, 4.315312551578999)

REZOLVARE

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_squared_error
from sklearn.datasets import make_regression

X, y = make_regression(n_samples = 1000, n_features = 1, noise = 2, bias = 3, rando
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta

ridge_reg = Ridge(alpha=0.2, random_state=42)
ridge_reg.fit(X_train, y_train)

print(ridge_reg.coef_, ridge_reg.intercept_)

y_pred = ridge_reg.predict(X_test)
err_val = mean_squared_error(y_test, y_pred)

print(err_val)
```

```
[16.73745169] 2.981569854879661
4.315312551578999
```

11. Use the following code to generate a regression dataset:

X, y = make_regression (
n_samples=1000,
n_features=2,
noise=1,
bias=2,
random_state=42
}

Split the data into a train and test split being 25% of the initial dataset.

Apply the polynomial kernel SVM algorithm with degree 2, coefficient 0.5, C=2 and ε = 1 for 5000 epochs on the train set.

Compute the second support vector and the mean squared error on the test set.

(array([0.11422765, 0.15030176]), 0.9818165401064261?)

REZOLVARE:

```
from sklearn.model_selection import train_test_split
from sklearn.svm import SVR
from sklearn.datasets import make_regression
import numpy as np
from sklearn.metrics import mean_squared_error

X, y = make_regression(n_samples = 1000, n_features = 2, noise = 1, bias = 2, rando
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_st

svm_reg = SVR(kernel="poly", degree=2, C=2, epsilon=1, coef0=0.5, max_iter=5000)
svm_reg.fit(X_train, y_train);

print(svm_reg.support_vectors_[1])
print(mean_squared_error(y_test, svm_reg.predict(X_test)))
```

```
[0.11422765 0.15030176]
0.9818168401064468
```

12. Use the following code to generate a regression dataset:

```
X, y = make_classification (
        n_samples=1000,
        n_features=4,
        n_informative=4,
        n_redundant=0,
        n_classes=3,
        n_clusters_per_class=1,
        random_state=42
}
```

Split the data into a train and test split being 25% of the initial dataset.
Apply the decision tree algorithm with a minimum of 5 samples in each leaf, on the train set.
Compute the feature importances, the accuracy on the test set and the output probability distribution for the 9th sample in the test set.

(array([0.15505521, 0.20428957, 0.35449143, 0.29316979]), 0.852, array([[0.33333333, 0.55555556, 0.11111111]])) (NU SUNT SIGURA)

REZOLVARE:

```
from sklearn.model_selection import train_test_split
from sklearn.datasets import make_classification
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
```

```
X, y = make_classification(n_samples = 1000, n_features = 4, n_redundant = 0, n_inf

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_st

tree_clf = DecisionTreeClassifier(min_samples_leaf=5, random_state=42)
tree_clf.fit(X_train, y_train)

print(tree_clf.feature_importances_)
print(accuracy_score(y_test, tree_clf.predict(X_test)))
print(tree_clf.predict_proba(X_test[8].reshape(1, -1)))
```

```
[0.15805821 0.20428957 0.35449143 0.28316079]
0.852
[[0.33333333 0.55555556 0.11111111]]
```

```
import os
from graphviz import Source
from sklearn.tree import export_graphviz

export_graphviz(
        tree_clf,
        out_file=os.path.join(".", "classification_tree.dot"),
        feature_names=["x[1]", "X[2]", "X[3]", "X[4]"],
        rounded=True,
        filled=True
    )

Source.from_file(os.path.join(".", "classification_tree.dot"))
```

13. Use the following code to generate a classification dataset:

```
X, y = make_circles (
        n_samples=1000,
        factors=0.5,
        noise=0.1,
        random_state=42
}
```

Split the data into a train and test split, the test split being 25% of the initial dataset.

Apply the K-means clustering algorithm with K=3 on the train set.

Compute the label of the 11th sample in the test set, the first cluster center, the inertia on the train set, the negative inertia on the test set, and the normalized mutual information score.
(2, array([-0.24530604, 0.53353481]), 186.76691886796457,

-58.722482875379455, 0.0023875255667463614)

REZOLVARE:

```
from sklearn.datasets import make_circles
from sklearn.model_selection import train_test_split
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
from sklearn.metrics.cluster import normalized_mutual_info_score
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.cluster import DBSCAN
from sklearn.cluster import AgglomerativeClustering

X, y = make_circles(n_samples = 1000, factor = 0.5, noise = 0.1, random_state = 42)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_st

kmeans = KMeans(n_clusters=3, random_state=42)
y_train_pred = kmeans.fit_predict(X_train)

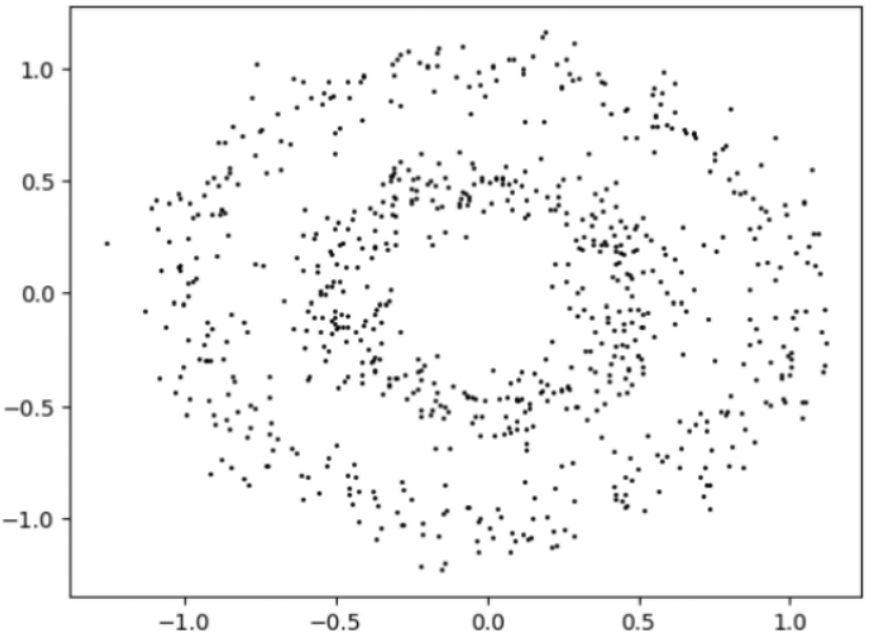
def plot_data(X):
    plt.plot(X[:, 0], X[:, 1], 'k.', markersize=2)

plot_data(X_train)

print(kmeans.labels_[10])
print(kmeans.cluster_centers_[0])
print(kmeans.inertia_)
print(kmeans.score(X_test))

print(normalized_mutual_info_score(y_test, kmeans.predict(X_test)))
```

```
/usr/local/lib/python3.9/dist-packages/sklearn/cluster/_kmeans.py:870: Futu
  warnings.warn(
2
[0.24530604 0.53353481]
186.75691886796457
-58.722482875379455
0.0023875255667463614
```



14. Use the following code to generate a classification dataset:

```
X, y = make_circles (
        n_samples=1000,
        factors=0.5,
        noise=0.1,
        random_state=42
}
```

Split the data into a train and test split, the test split being 25% of the initial dataset.

Apply the K-means clustering algorithm with K=5 and a maximum of 10 iterations, on the train set.

Compute the label of the 5th sample in the test set, the first cluster center, the inertia on the train set, the negative inertia on the test set, and the silhouette score.

(0, array([-0.21019163, 0.67227348]), 104.42381022725004, -33.16508490326328, 0.08178833442790581)

REZOLVARE:

```
from sklearn.datasets import make_circles
from sklearn.model_selection import train_test_split
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

X, y = make_circles(n_samples = 1000, factor = 0.5, noise = 0.1, random_state = 42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.25, random_

kmeans = KMeans(n_clusters=5, random_state=42, max_iter = 10)
y_train_pred = kmeans.fit_predict(X_train)

print(kmeans.labels_[4])
print(kmeans.cluster_centers_[2])
print(kmeans.inertia_)
print(kmeans.score(X_test))

print(silhouette_score(X_test, y_test))
```

```
0
[-0.21019163  0.67227348]
104.42381022725004
-33.16508490326328
0.08178833442790581
/usr/local/lib/python3.9/dist-packages/sklearn/cluster/_kmeans.py:870: FutureW
  warnings.warn(
```

1. Use the following code to generate a regression dataset:

```
X, y = make_regression (
        n_samples=1000,
        n_features=1,
        noise=1,
        bias=2,
        random_state=42
}
```

Split the data into a train and test split being 20% of the initial dataset.
Apply the stochastic gradient descent regression algorithm with no regularization for 100 epochs with a constant learning rate on the train set.
n=0.005 on the train set
Compute w, w0, and the mean squared error on the test set.
(array([16.74232235]), array([1.98795322]), 1.0791838748150104)

REZOLVARE:

```
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split
from sklearn.linear_model import SGDRegressor
from sklearn.metrics import mean_squared_error

X, y = make_regression(n_samples = 1000, n_features = 1, noise = 1, bias = 2, rando
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_s

sgd_reg = SGDRegressor(max_iter=100, learning_rate='constant', random_state=42, pen
sgd_reg.fit(X_train, y_train)

print(sgd_reg.coef_, sgd_reg.intercept_)

y_pred = sgd_reg.predict(X_test)
print(mean_squared_error(y_test, y_pred))
```

```
[16.60857971] [2.02087757]
1.084387800319569
```